

Problem 23 24689126

Intended difficulty: Hard

The problem I propose is can be solved in a few different ways but the intended solution will be either Kahn's topological sort + earliest finish dynamic programming, or, DFS with memoisation to find the earliest finish times with a backward pass to count the critical checks.

This is a graph problem where we can model the problem with checks as vertices and the dependencies as directed edges. The problem is about finding the longest path in a weighted directed acyclic graph with each node having a time as weight. We can evaluate a graph path as $\text{time}[A] + \text{time}[B] + \dots + \text{time}[N]$. However, the problem allows checks on separate branches to run in parallel with a check waiting for the latest finishing prerequisite.

The dynamic aspect comes from tracking the earliest finishing time for each check using the equation. $F(v)$ is v 's earliest finish time and T is the launch completion time.

$$F(v) = \begin{cases} t(v) & \text{if } v \text{ has no prerequisites} \\ t(v) + \max\{F(u) : u \text{ is a prerequisite of } v\}, & \text{Otherwise} \end{cases} \quad T = \max_{v \in V} F(v)$$

Kahn's topological sort. We can initialise a queue with checks with no prerequisites or in degree. When processing a check 'u' we can update each dependent check, update its in degree and enqueue when a check no longer has prerequisites. Every prerequisite is processed before the dependents so each $F(v)$ is final when used.

DFS with memoisation. For each uncalculated check, follow its prerequisites first, then calculates its finish time from the prerequisites and stores it.

Backwards path. The latest each check can finish without delaying the launch. A check must finish before every check after it needs to start. We can achieve this by processing checks in reverse topological order and comparing each checks earliest finish to its latest finish. We can count values that are equal. Let $L(v)$ be the latest finish time that still allows the launch to finish by T . $C(v)$, let 'v' be a critical check.

$$L(v) = \begin{cases} T, & \text{if } v \text{ has no successor} \\ \min\{L(w) - t(w) : v \rightarrow w\}, & \text{Otherwise} \end{cases}$$

$$C(v) \text{ iff } F(v) = L(v)$$

Using these combine to be the Critical Path Method (CPM) however, application of techniques and methods found in the lecture are used to achieve it.

Time complexity:

The intended solution uses Kahn's queue based topological sorting.

- Building adjacency list and calculating in-degrees: $O(N + M)$
- Initial loop: $O(N)$
- Graph traversal: $O(N + M)$
- Backward Pass: $O(N + M)$

Overall: $O(N + M)$

Space complexity:

- Adjacency list: $O(M)$
- In-degree list: $O(N)$
- Queue: $O(N)$ worst case

- Dynamic programming array: $O(N)$

Overall: $O(N + M)$

AI Declaration.

This project has been modelled after a well known problem. I have mostly used the resources in references to model solutions and ideas. AI was mostly used to provide validation for correctness of inputs and subsequently correctness of solutions. It also helped provide extended insight on various approaches and exploration of edge cases to help improve testing.

References

Er, M. C. (1983). A parallel computation approach to topological sorting. *The Computer Journal*, 26(4), 293–295. <https://doi.org/10.1093/comjnl/26.4.293>

Carnegie Mellon University, Department of Civil and Environmental Engineering. (n.d.). 10. *Fundamental scheduling procedures*. In *Project management for construction*.
https://www.cmu.edu/cee/projects/PMbook/10_Fundamental_Scheduling_Procedures.html

Sedgewick, R., & Wayne, K. (n.d.). 4.4 *Shortest paths*. In *Algorithms, 4th edition*. Princeton University.
<https://algs4.cs.princeton.edu/44sp>

LeetCode. (n.d.). 2050. *Parallel Courses III*. Retrieved October 2, 2026, from
<https://leetcode.com/problems/parallel-courses-iii/description/>